Birla Institute of Technology & Science, Pilani Hyderabad Campus

CS F372: Operating Systems

Assigned: 22/09/26 **Date of Submission: 14/10/26**

Total Marks: 12 **Weightage: 6%**

General Instructions:

- This assignment can be done in groups of **maximum 4** students.
- All your programs must run on a **Ubuntu 24.X virtual machine on Oracle Virtual Box**.
- Recommended allocation: 4 cores, 50 GB virtual disk, 4GB RAM or more on VM.
- Submit all source files and executables along with a readme that has group details (Names and ID numbers of all members of the group.)
- Put all your deliverables in a zip/tar file and submit through the course webpage (assignment section). One submission per group is sufficient.
- Every assignment will have a demo and viva, the schedule of which will be intimated later through a separate notice.

For any assignment related queries, you may contact Instructors or the following teaching assistants:

- Arpit Kudre <f20230158@hyderabad.bits-pilani.ac.in>
- Parth Munjal <f20230058@hyderabad.bits-pilani.ac.in>

Problem 1: Multithreaded Student Report Card Generator **[2 Marks]**

Write a multi-threaded POSIX compliant C program that aggregates academic data per student, sorts students by a user-specified criterion, and generates report cards through an interactive menu.

Input and Marks Matrix:

The program must read the marks matrix from the input.txt file. The file format is as follows.

```
4   //number of subjects (i.e. rows)
6   //number of students (i.e. columns)
60 88 55 91 63 79 // (i,j) represents marks of jth student in ith subject
70 90 60 85 70 82
66 77 58 95 68 75
71 85 62 89 66 80
```

Further input is to be taken from the command line through stdin.

Phase 1 - Aggregation:
The parent thread spawns one worker thread per student. *Worker[i]* reads *column[i]* from the marks matrix and fills in *student[i]'s* entry: student_id, average, minimum, maximum, and a copy of their full mark list in **subject_marks[]** as shown in the struct below. Each worker exits via **pthread_exit()** once it's done, the parent is responsible for joining them before moving to Phase 2.

```
typedef struct {
    int student_id;
    double average;
    int minimum;
    int maximum;
    int subject_marks[MAX_SUBJECTS];  //MAX_SUBJECTS: 32, MAX_STUDENTS: 128
} student_stats;
```

The result of this phase is an array of **student_stats**, which contains one struct entry per student, indexed by **student_id**.

Phase 2 - Sorting:
The parent prompts the user for two values, **the sort key** which dictates which parameter will be used for sorting (**-3**=max, **-2**=min, **-1**=avg, **0..num_subjects-1**=subject index), and **k**, which is the number of sorting threads (1 ≤ k ≤ num_students, assume it is also a factor of **num_students**).

Threading Model:
Worker Threads:
Each of the k sorting threads must:

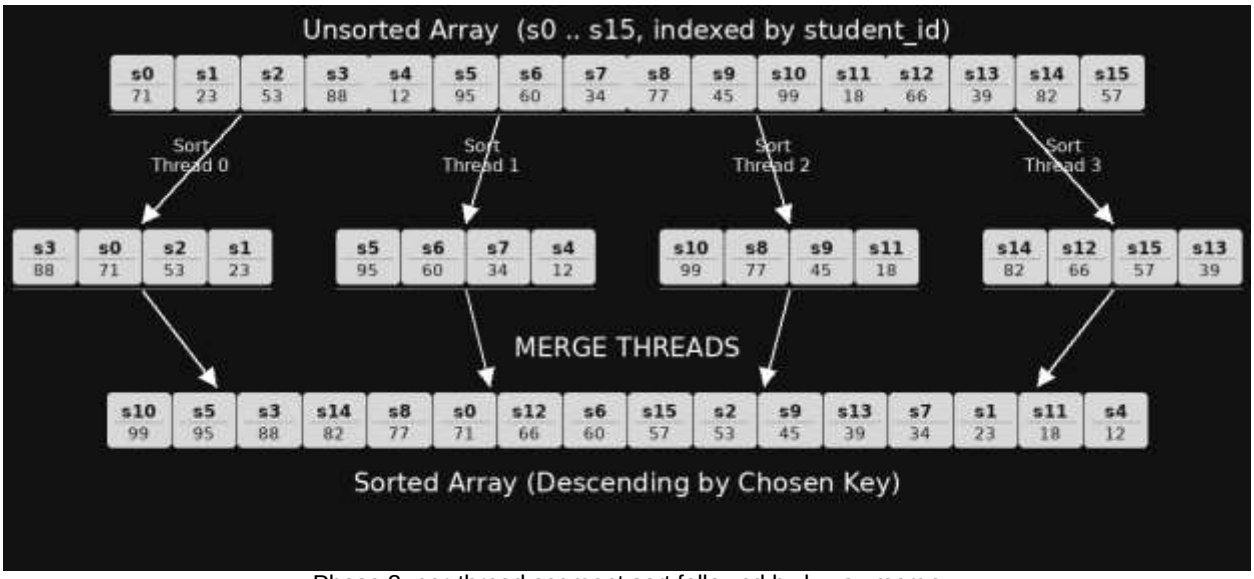
1. Sort its assigned segment of the **student_stats** array in-place, in descending order according to the chosen sort key, using any sorting algorithm of your choice. A thread must not modify elements outside its assigned segment.
2. Break ties (two students equal on the chosen key) by **student_id**, with the lower id placed first. This rule must be applied by the worker and parent threads.
3. Terminate after completing its work via **pthread_exit()**.

Parent Thread:
The main parent thread must:

1. Divide the **student_stats** array into k contiguous segments and create the k worker threads, passing each the following struct as its argument.

```
struct thread_params {
   int thread_id;              // Logical thread ID
   int start_index;            // Starting index in student_stats array
   int segment_size;           // Number of elements to process
};
```

2. Wait for all k threads to complete via **pthread_join()**.
3. Perform a **k-way merge** of the k sorted segments into a new array of the same size, containing all students in a single globally **descending order** on the chosen key.



Phase 2: per-thread segment sort followed by k-way merge

Phase 3 - User Queries

No threads are spawned here. The parent reads from stdin in a loop:

- **1 <student_id>** : print that student's id, average, minimum, maximum, and full **subject_marks[]**
- **2** : print the current sorted order (without re-sorting) as: student ids, then their value on the active sort key.
- **0** : exit

Sample Output:

```
Enter sort key (-3=max, -2=min, -1=avg, 0..3=subject index): -1
Enter number of sorting threads (k): 3
--- Report Card Menu ---
Enter command (1 <id> = view student, 2 = view all, 0 = exit): 1 2
Student ID    : 2
Average       : 58.75
Minimum       : 55
Maximum       : 62
Subject Marks: [55, 60, 58, 62]
Enter command (1 <id> = view student, 2 = view all, 0 = exit): 2
s3    90.00
s1    85.00
s5    79.00
s0    66.75
s4    66.75
s2    58.75
Enter command (1 <id> = view student, 2 = view all, 0 = exit): 0
```

NOTE:

- **You may use pthread_create(), pthread_join(), pthread_exit(), pthread_self(), pthread_detach(), and pthread_cancel() to implement this.**
- **Do NOT use any synchronization primitives such as pthread_mutex_lock(), pthread_mutex_unlock(), semaphores (sem_wait, sem_post), or condition variables.**

Problem 2: Real-Time Scheduling: Round-Robin vs FIFO **[3 Marks]**

Objective:

To observe the relationship between the Round-Robin time quantum, preemption and context switching under CPU-Bound and I/O-bound workload characteristics, and to compare this behavior against the FIFO real-time scheduling policy and to measure the effect of both policies on response time, turnaround time and throughput.

The Question:

- Preparation: Modify the system-wide round-robin timeslice to 6 different values: 1ms, 10ms, 50ms, 150ms, 300ms and 500ms.
  HINT: You will need to change the **sched_rr_timeslice_ms** kernel parameter.
  Only one run will be required for the FIFO scheduling policy.

- Implementation: Write two C programs. Each program must fork 8 child processes.
  - Program A (CPU-Bound): Each child process must execute the task provided in **workload_cpu.h**.
  - Program B (I/O-Bound): Each child process must execute the task provided in **workload_io.h**.

- The parent records one reference timestamp (**clock_gettime(CLOCK_MONOTONIC)**) before forking. Each child records its own start (first action after **fork()** returns) and completion (before it exits) timestamps, and reports both to the parent via a pipe created before **fork()**. The parent computes, per child, using its own reference timestamp:
  - Response Time = child's start timestamp − reference timestamp
  - Turnaround Time = child's completion timestamp − reference timestamp

- Execution: To ensure valid execution, you must force all 8 processes to compete for the same CPU core. If they run on different cores, they will not preempt each other frequently, leading to incorrect or misleading results.

  For the **round-robin runs**, execute your programs with round-robin scheduling using the SCHED_RR real-time policy:
  Execution command: **sudo taskset -c 0 chrt -r <priority> ./program_name**

  For the **FIFO run**, execute your programs using the SCHED_FIFO real-time policy instead:
  Execution command: **sudo timeout 30 taskset -c 0 chrt -f <priority> ./program_name**

Explanation:

- **taskset -c 0** : Pins all processes to CPU core 0
- **chrt -r <priority>** : Sets SCHED_RR scheduling policy with priority (recommended: 10-50)
- **chrt -f <priority>** : Sets SCHED_FIFO scheduling policy with priority (recommended: 10-50)
- **timeout 30** : Caps the FIFO run at 30 seconds. SCHED_FIFO has no forced preemption, so a CPU-bound run that does not terminate on its own will not yield the core.

- Data Collection and Submission: For each of the 6 time quantum values, and once more under SCHED_FIFO, and for both programs, record the following output:
  - Involuntary Context Switches (aggregated across all children)
  - Voluntary Context Switches (aggregated across all children)
  - Average Response Time (across all children)
  - Maximum Response Time (across all children)
  - Average Turnaround Time (across all children)
  - Maximum Turnaround Time (across all children)
  - Throughput (number of children / Total Elapsed Time)
  - Total Elapsed Time (measured by the parent process)

Your parent program may use **getrusage(RUSAGE_CHILDREN, &usage)** to retrieve aggregated context switch information for all terminated child processes. Make sure to submit a **.txt file** along with the C programs containing the aforementioned statistics.

Sample Output:

```
Program-A (CPU-Bound)
--------------------------------------------------
Time Quantum: 100ms (For Illustration only)
  Involuntary Context Switches: 42
  Voluntary Context Switches:   8
  Average Response Time:        350.28ms
  Maximum Response Time:        700.20ms
  Average Turnaround Time:      2501.60ms
  Maximum Turnaround Time:      2572.53ms
  Throughput:                   3.11 children/s
  Total Elapsed Time:           2.57s
[....continue for all the time quantum values (1, 10, 50, 150, 300,
500)....]

Scheduling Policy: SCHED_FIFO
  [...Same parameters as above...]
Program-B (I/O-Bound)
--------------------------------------------------
[Same format as above]
```

Problem 3: Sudoku Conflict Analyzer and Repair Advisor **[4 Marks]**

Objective:

Design a multi-threaded, POSIX-compliant C program that validates a 9×9 Sudoku grid and, when the grid is wrong, produces analytics that localise the error down to individual cells and name the digits needed to repair each broken unit.

Rules of the game:

Every cell holds a number from 1 to 9. Every row, every column and each of the nine 3×3 sub-grids must contain each of 1–9 exactly once. A cell may also be empty, written as 0 or . in the input file; an empty cell makes its row, its column and its sub-grid invalid.

Input:

The grid is read from a file whose path is passed as argv[1] — it is not hard-coded. The file contains 81 cell values in row-major order. Digits may be separated by spaces, newlines or nothing at all; 0 and . both denote an empty cell. Reject the input with a diagnostic if fewer or more than 81 cells are present.

$ ./sudoku_analyzer grid_invalid.txt

Phase 1 — Validation

The parent spawns 27 validation worker threads: 9 for rows, 9 for columns, 9 for sub-grids. Thread unit_id 0–8 are rows 0–8, 9–17 are columns 0–8, and 18–26 are sub-grids 0–8 in row-major order.

Each worker receives a pointer to:

```c
struct unit_params {
    int unit_id; /* 0-26 */
};
```

and must, for its own unit:

1. Count the occurrences of each digit 1–9 and of empty cells.
2. Write its verdict to validity[unit_id] in the global 27-element validity array (VALID = 1, INVALID = 0). A thread writes only its own index.
3. Fill in a heap-allocated result struct and return it to the parent via pthread_exit():

```c
struct unit_result {
    int unit_id;
    int valid;          /* 1 = VALID, 0 = INVALID */
    int missing_mask;   /* bit d set  => digit d never appears */
    int dup_mask;       /* bit d set  => digit d appears twice or more*/
    int blank_count;    /* number of empty cells in this unit */
};
```

The parent collects all 27 pointers with pthread_join() and prints VALID or INVALID. If the grid is valid the program prints VALID, frees its allocations and exits.

Phase 2 — Analytics

Enter only when the grid is invalid.

(a) Validity Score

The parent prints the percentage of the 27 units that reported valid, to two decimal places.

(b) Conflict Density Heatmap

The parent spawns 9 analytics worker threads, one per 3×3 block of the grid (not one per row — each thread owns the nine cells of its own block and writes nowhere else):

```c
struct block_params {
    int   block_id;                 /*   0-8,   row-major   */
};
```

Each thread fills the 9 cells of its block in the global 9×9 heatmap. Unlike a per-unit error flag, the score is per cell and blames only the cells that actually take part in a duplication:

Conflict score of a cell holding value v = (v is duplicated in its row) + (v is duplicated in its column) + (v is duplicated in its sub-grid)

An empty cell scores 3 outright.

Each term is 0 or 1, so scores run 0–3. A cell scores 0 if its value is unique in all three of its units, even when those units are themselves invalid for some other reason. A cell scoring 3 is a suspected epicentre.

The duplication information comes from the dup_mask fields the parent already gathered in Phase 1. The parent copies them into a global array before creating the analytics threads; that array, like the validity array, is read-only with respect to the analytics threads.

Print the heatmap as a 9×9 grid of digits.

(c) Hotspot Report

List every cell with score 3 as (row, col) value=v, in row-major order, followed by the total count.

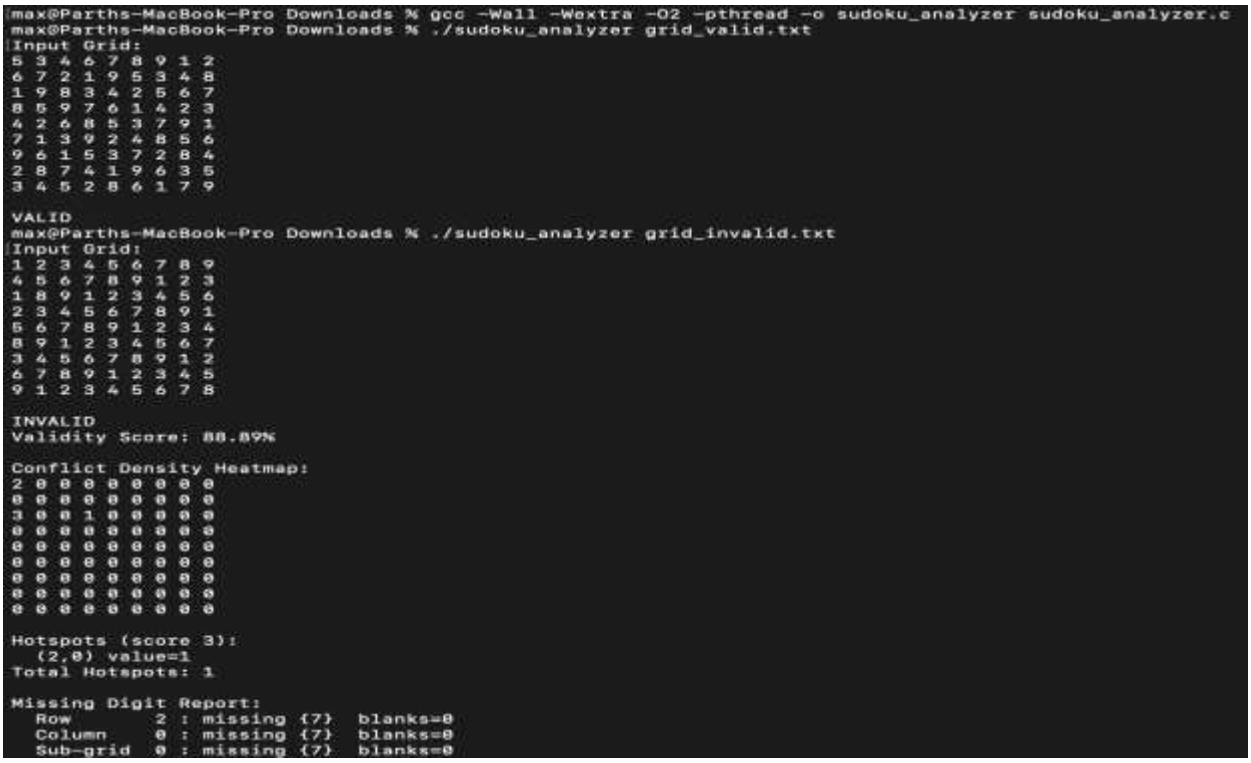
(d) Missing Digit Report

For each of the 27 units that was invalid, print the unit's kind and index, the set of digits missing from it, and its blank count. This is the "repair advisor": for a unit with exactly one blank and exactly one missing digit, the report names the digit that belongs in that cell.

Required output order

1. The input grid echoed.
2. VALID or INVALID.
3. Validity score.
4. Conflict density heatmap.
5. Hotspot report.
6. Missing digit report.

Sample run



Notes

- You may use pthread_create(), pthread_join(), pthread_exit(), pthread_self(), pthread_detach() and pthread_cancel().
- Do NOT use any synchronization primitives — no pthread_mutex_lock()/ pthread_mutex_unlock(), no semaphores (sem_wait, sem_post), no condition variables. Your design must be race-free by *partitioning*, and you will be asked at the viva to justify why each shared array is safe.
- Every struct returned through pthread_exit() must be malloc()ed by the worker and free()d by the parent. Marks are deducted for leaks; valgrind --leak-check=full must be clean.
- Build with gcc -Wall -Wextra -O2 -pthread. Warnings count against you.

Problem 4: Fork Storm Containment & Tiered CPU Scheduling [3 Marks]

The Scenario:

You are a kernel developer for a shared computer cluster. Two problems keep recurring:

1. **Runaway workloads.** Users launch fork bombs, deliberately or by accident, and the node becomes unusable.
2. **Differentiated service.** The cluster owner has sold three service levels — gold, silver and bronze — and wants CPU share to reflect them.

Your job is to modify the Linux kernel (v6.x) to add a **graduated fork-rate containment system** and a **tiered CPU scheduling policy**, such that performance privileges never override safety policy.

Objective:

Implement two orthogonal kernel mechanisms that must coexist without interfering with each other.

Mechanism 1 — Fork Storm Containment (escalation ladder)

Last year's design killed an offender on sight. This year the response must be **graduated**, so that a merely bursty but legitimate workload is slowed rather than destroyed. Your kernel must:

- Monitor process creation frequency **per user**, over a configurable sliding window.
- Escalate through three stages when the threshold is exceeded within one window:
    - **Stage 1 — THROTTLE.** Refuse the fork with -EAGAIN and impose a configurable penalty delay. Log the event.

    - **Stage 2 — FREEZE.** The user is still over the threshold: send SIGSTOP to the offending process group and log it. The group is suspended, not destroyed, so it can be inspected.
    - **Stage 3 — REAP.** Still over the threshold: SIGKILL the offending process group and log it.
- **De-escalate.** An offender whose fork rate returns to normal for longer than a configurable cooldown is forgiven and returns to stage 0.
- Exempt the **root** user entirely, to preserve system recovery.
- Restore shell control to the user once the offending group is gone.
- Log every detection and every stage transition through the kernel log (dmesg).
- Expose **all** thresholds through sysctl, changeable without recompiling: at minimum an enable switch, the fork threshold, the window length, the throttle delay and the cooldown period.

Mechanism 2 — Tiered CPU Scheduling

Replace the single "VIP" notion with **three named service tiers**. Your kernel must:

- Support **GOLD**, **SILVER** (the default for everyone) and **BRONZE**, assigned per user.
- Allow **only root** to assign tiers, through a custom command — tier set <username> <gold|silver|bronze> — and to clear all assignments with tier reset.
- Permit **at most one GOLD user at a time**; assigning GOLD to a second user automatically demotes the incumbent to SILVER.
- Give a GOLD user's processes a visibly larger CPU share and a BRONZE user's a visibly smaller one, by biasing the task's CFS weight.
- **Re-tier processes that are already running.** Promoting a user whose CPU-bound job is already in flight must take effect immediately, without restarting the job.
- Make children inherit their user's tier.
- **Prevent starvation.** The tier bias must be clamped so that a BRONZE user competing against a GOLD user still makes measurable progress. Justify your clamp at the viva.
- Ensure tier privilege does **NOT** bypass fork containment.

Mechanism 3 — Observability

Expose a read-only /proc file reporting, per user: current tier, forks counted in the current window, current escalation stage, and cumulative counts of throttle, freeze and reap events. Also expose the live values of every tunable. Your tier show command must be able to display it.

Expected behaviour

Fork storm, stock kernel: the system becomes temporarily unresponsive, fork attempts fail repeatedly, and the user must kill the terminal by hand.

Fork storm, your kernel: a short pause, then a visible escalation in dmesg — throttle, then freeze, then reap — the offending group dies, and the shell prompt comes back on its own. *If you have to kill the terminal manually, containment has failed.*

Tiers: two CPU-bound users at SILVER share a core roughly 50/50. Promote one to GOLD and its share rises visibly and immediately. Demote the other to BRONZE and the gap widens further, but the BRONZE user must still accumulate CPU time.

Tier + fork storm: a GOLD user's fork bomb is contained exactly like anyone else's. Safety overrides privilege.

A detailed help document covering the kernel build, the specific functions to hook and the required /proc and sysctl plumbing accompanies this assignment.

Test Cases

The machine must have at least two accounts: one with root privileges and one ordinary user.

1. **Baseline.** Run taskset -c 0 yes > /dev/null & as both users and watch top. Expect a roughly 50/50 split.



2. **Promote.** From the root terminal, with both workloads and top still running, run tier set <user> gold. The promoted user's share must rise **without restarting the workload**.

```
root@vm — promote

root@vm:~# tier set alice gold
user alice (uid 1001) -> gold

root@vm — top (a few seconds later)

  PID USER    PR  NI  %CPU  %MEM  TIME+   COMMAND
 4821 alice   13  -7  84.7   0.0  1:02.11 yes
 4830 bob     20   0  14.9   0.0  0:41.02 yes

root@vm — proof it's the same process, not a restart

root@vm:~# ps -u alice -o pid,ni,pcpu,etime,comm
    PID  NI %CPU     ELAPSED COMMAND
   4821  -7 84.6       01:47 yes
(same PID and ELAPSED as before promotion — it kept running the whole time)
```

3. **Demote.** Set the *other* user to bronze. The gap widens. Confirm from top that the bronze user still accumulates CPU time — it must not be starved to zero.

```
root@vm — demote

root@vm:~# tier set bob bronze
user bob (uid 1002) -> bronze

root@vm — top

  PID USER    PR  NI  %CPU  %MEM  TIME+   COMMAND
 4821 alice   13  -7  91.2   0.0  2:15.88 yes
 4830 bob     25   5   6.1   0.0  0:58.40 yes

root@vm — tier show

root@vm:~# tier show
UID      TIER     NICE
1001     GOLD     -7
1002     BRONZE   5
```

4. **Reset.** Run tier reset and confirm the split returns to roughly 50/50.

```
root@vm — reset

root@vm:~# tier reset
all users reset to SILVER

root@vm — top (a few seconds later)

  PID USER    PR  NI  %CPU  %MEM  TIME+   COMMAND
 4821 alice   20   0  48.9   0.0  2:41.05 yes
 4830 bob     20   0  50.6   0.0  1:24.90 yes
```

5. **Containment, untiered.** Run sudo forkstat on the root terminal, then launch :(){ :|:& };: as the ordinary user. Watch the ladder run: the bomb must stop by itself and the shell must come back.



6. **Containment, GOLD.** Repeat test 5 with the offending user crowned GOLD first. The outcome must be identical.



7. **Retune at runtime.** Lower fg_threshold via sysctl, repeat test 5, and show that containment triggers sooner — with **no kernel rebuild**.

```
root@vm — lower the bar

root@vm:~# sysctl -w kernel.fg_threshold=50
kernel.fg_threshold = 50
root@vm:~# sysctl -w kernel.fg_window_ms=500
kernel.fg_window_ms = 500

root@vm — bomb triggers noticeably sooner

[ 1002.881 ] fork_guard: THROTTLE uid=1001 pgid=3701 (>50 forks in 500 ms)
(compare: test 5 needed >200 forks in 1000 ms before the first warning)

root@vm — restore normal settings

root@vm:~# sysctl -w kernel.fg_threshold=200
root@vm:~# sysctl -w kernel.fg_window_ms=1000
```

8. **Observability.** cat your /proc stats file and show the escalation counters and tier assignments. Then check dmesg for the fork-containment and tier log lines.

```
root@vm — /proc/fork_guard

root@vm:~# cat /proc/fork_guard
enabled=1 threshold=200 window_ms=1000 throttle_ms=40 cooldown_s=10

UID      TIER     FORKS    STAGE    THROTTLE  FREEZE  REAP
1001     GOLD     0        0        2         2       2
1002     BRONZE   0        0        0         0       0

root@vm — /proc/sched_tier

root@vm:~# cat /proc/sched_tier
UID      TIER     NICE
1001     GOLD     -7
1002     BRONZE   5

root@vm — kernel boot confirmation

root@vm:~# uname -r
6.8.0-forkguard
root@vm:~# dmesg | grep -m1 fork_guard
[    1.902144] fork_guard: initialised (threshold=200/1000ms, gold nice=-7)
```
